

Tres proyectos, después de VitaLink

Matías Acosta · 27 de septiembre de 2026

VitaLink cubre mucho: integración con un sistema viejo, un modelo de dominio con reglas clínicas verificadas, una base relacional, una API REST, una interfaz, infraestructura como código y 358 pruebas automatizadas. Pero deja agujeros, y estos tres proyectos están elegidos por los agujeros que tapan, no por el tema.

Cada uno usa el mismo juego de herramientas —TypeScript, Prisma, PostgreSQL, Docker, Terraform, React— para que lo que ya sabés no se vuelva a aprender, y tiene **un problema difícil en el centro**: eso es lo que después se cuenta en una entrevista. Un proyecto sin un problema difícil es una demostración de que sabés seguir un tutorial.

Están en el orden en que los haría. El primero tapa el agujero que más te van a hacer notar.

Sistema de turnos con autenticación y roles

El agujero: VitaLink no tiene usuarios. Ni login, ni permisos, ni sesiones. Es lo primero que pregunta cualquier entrevistador de backend, y hoy no tenés con qué contestar.

QUÉ ES

Una agenda de turnos con tres roles: el paciente que reserva, el profesional que ve su agenda y define su disponibilidad, y el administrativo que ve todo y puede reasignar. Registro, inicio de sesión, recuperación de contraseña, y cada pantalla mostrando sólo lo que ese rol puede ver.

El dominio lo podés contar con la misma autoridad que VitaLink: sabés cómo funciona una agenda de verdad, cuándo se sobretura, qué pasa cuando alguien no viene.

EL PROBLEMA DIFÍCIL

Dos personas reservando el mismo turno en el mismo instante.

La solución que todo el mundo escribe primero es: consultar si el turno está libre y, si lo está, insertarlo. Está mal, y falla justo cuando el sistema se usa — entre la consulta y la inserción hay una ventana, y en esa ventana entra el otro. Se resuelve con una restricción de unicidad en la base y una transacción, dejando que PostgreSQL rechace al segundo en vez de preguntarle primero.

Es de la misma familia que la idempotencia de VitaLink, pero del lado de la escritura concurrente: allá el problema era *repetir*, acá es *competir*.

LO QUE APRENDÉS

- Autenticación con sesiones o tokens, y por qué el *refresh token* existe
- Autorización por rol, aplicada en el servidor y no sólo escondiendo botones
- Hash de contraseñas, y por qué nunca se compara con ===
- Transacciones, restricciones de unicidad y condiciones de carrera
- Envío de correo transaccional: confirmación, recordatorio, cancelación

CÓMO SABÉS QUE TERMINASTE

- Un test que lanza dos reservas simultáneas al mismo turno y comprueba que exactamente una gana
- Un test que pide un recurso ajeno con el token de otro usuario y recibe 403, no 200
- La contraseña no aparece en ninguna respuesta, en ningún log, en ninguna traza de error

Stack TypeScript · NestJS · Prisma · PostgreSQL · React · Docker

Tamaño El más grande de los tres. Dos o tres semanas a ritmo tranquilo.

Ingesta con cola, reintentos y cola de mensajes muertos

El agujero: VitaLink hace todo de forma sincrónica. Un POST /sync que procesa y devuelve. Eso no escala, y cuando algo falla a la mitad no hay a dónde volver.

QUÉ ES

Un servicio que recibe eventos —lecturas de sensores, pedidos, registros de actividad, lo que elijas— los deja en una cola, y un consumidor los procesa de a lotes, por su cuenta, sin que nadie espere. En AWS eso es SQS más una Lambda; en tu máquina, un contenedor más.

Es el patrón que separa un proyecto de práctica de un sistema real: el que produce y el que consume dejan de estar atados, y el sistema empieza a tolerar que una parte se caiga.

EL PROBLEMA DIFÍCIL

El mensaje que falla siempre.

Llega uno malformado, o que dispara un error que no previste. El consumidor falla, la cola se lo devuelve, vuelve a fallar. Sin una decisión explícita, ese mensaje bloquea la cola para siempre o gira eternamente quemando dinero.

Hay que decidir cuántas veces se reintentará, cuánto se espera entre intentos —creciendo, no siempre igual—, y a dónde va el que nunca se pudo procesar. Eso último es una *dead-letter queue*, y lo importante no es configurarla: es que alguien se entere de que tiene algo adentro.

LO QUE APRENDÉS

- Colas, consumidores y procesamiento por lotes
- Reintentos con espera creciente, y por qué el reintento inmediato empeora una caída
- Consumidores idempotentes — un mensaje puede llegar dos veces, siempre
- Colas de mensajes muertos y alarmas sobre ellas
- Observabilidad: métricas de profundidad de cola, antigüedad del mensaje más viejo, tasa de error

CÓMO SABÉS QUE TERMINASTE

- Podés apagar el consumidor diez minutos, prenderlo, y no se perdió ni se duplicó nada
- Un mensaje envenenado termina en la cola de muertos después de N intentos y dispara una alarma
- Un panel muestra cuántos mensajes esperan y hace cuánto llegó el más viejo

Stack TypeScript · AWS SQS y Lambda · Terraform · Prisma · PostgreSQL · Docker

Tamaño El más chico de los tres, y el que más impresiona por línea de código.

Cobros con pasarela de pago y webhooks

El agujero: nunca integraste un sistema del que no controlás el tiempo. En Vitalink vos decidís cuándo pasa todo. Acá no: el otro sistema te avisa cuando quiere.

QUÉ ES

Un flujo de pago completo, de punta a punta, en modo de prueba del proveedor: el usuario inicia un cobro, lo completa fuera de tu aplicación, y minutos después el proveedor te avisa el resultado por un *webhook* — una petición HTTP que llega a tu servidor sin que la hayas pedido.

No hace falta mover un peso: todas las pasarelas tienen claves y tarjetas de prueba publicadas justamente para esto.

EL PROBLEMA DIFÍCIL

El webhook que llega dos veces, desordenado, o de alguien que se hace pasar por el proveedor.

Tu endpoint de webhook es público: cualquiera puede mandarle un JSON que diga "este pago se aprobó". Por eso el proveedor firma cada aviso, y verificar esa firma *antes* de leer el cuerpo es lo primero que hay que hacer.

Y aunque sea legítimo, puede llegar repetido — las pasarelas reintentan si no contestás rápido — o desordenado, con el "pago aprobado" llegando después del "pago reembolsado". Un identificador de evento guardado y una máquina de estados que no retrocede resuelven las dos cosas. Es tu lección de idempotencia otra vez, ahora con plata del otro lado.

LO QUE APRENDÉS

- Verificación de firma criptográfica de un webhook, y por qué se hace sobre el cuerpo crudo
- Claves de idempotencia y deduplicación de eventos
- Máquinas de estado para algo que no puede retroceder
- Reconciliación: qué hacer cuando tu base y el proveedor no coinciden
- Exponer un endpoint público de forma segura, que es lo que a Vitalink le falta

CÓMO SABÉS QUE TERMINASTE

- Un webhook con firma inválida se rechaza con 401 y queda registrado
- El mismo evento enviado tres veces deja exactamente un cobro en la base
- Un "aprobado" que llega después de un "reembolsado" no revive el cobro

Stack TypeScript · NestJS · Prisma · PostgreSQL · React · Docker · pasarela en modo prueba

Tamaño Mediano. El flujo es corto; la corrección es lo que lleva tiempo.

Una regla que vale para los tres, y que ya probaste que funciona: **escribí la decisión cuando la tomás, con el número al lado**. Las ADR de Vitalink son hoy lo que te permite defender ese proyecto sin recordarlo entero. Empezá cada uno de estos con una carpeta docs/decisions/ vacía y llenala sobre la marcha.